

Analisi e progettazione del software:

Che cos'è il software?

Un software è un programma per computer e la documentazione ad esso associata. I prodotti software possono essere:

- Generici: sviluppati per un ampio insieme di clienti.
- Personalizzati: sviluppati per un singolo cliente in base alle sue esigenze.

Un nuovo software può essere creato sviluppando nuovi programmi, personalizzando sistemi software pre-esistenti o riusando il software esistente.

Caratteristiche del software:

Un software deve avere le seguenti caratteristiche:

- Mantenibilità;
- Fidatezza;
- Efficienza;
- Accessibilità.

Ingegneria del software:

L'ingegneria del software è una disciplina ingegneristica che si occupa di tutti gli aspetti della produzione del software di buona qualità, dalle prime fasi della specifica del sistema fino alla manutenzione del sistema dopo la sua messa in uso.

Processo software:

Un processo software è un approccio disciplinato per la costruzione, il rilascio e la manutenzione del codice. Definisce chi (ruoli) fa cosa (attività), quando (organizzazione temporale) e come (metodologie) per raggiungere un certo obiettivo. Vi sono vari tipi di processi per lo sviluppo, ma la maggior parte hanno (o rielaborano) le attività fondamentali.

Attività fondamentali di un processo software:

- Requisiti;
- Analisi;
- Progettazione;
- Implementazione;
- Validazione;
- Rilascio e installazione;
- Manutenzione ed evoluzione;
- Gestione del progetto.

Analisi e progettazione:

Sono due attività di processo fondamentali:

- **Analisi:** enfatizza un'investigazione di un problema e dei suoi requisiti.
- **Progettazione:** enfatizza una soluzione concettuale che soddisfa i requisiti del problema.

In particolare, nell'ambito object oriented:

- **Analisi orientata agli oggetti:** definisce le classi concettuali (di dominio); servono a descrivere i concetti o gli oggetti (del mondo reale) relativi al problema;
- **Progettazione orientata agli oggetti:** definisce le classi software, che servono a modellare i precedenti concetti del mondo reale in classi e oggetti software utilizzabili all'interno di codice scritto in un linguaggio Object Oriented.

L'analisi orientata agli oggetti è interessata alla creazione di una descrizione del dominio da un punto di vista ad oggetti. Vengono identificati i concetti, gli attributi e le associazioni considerati significativi. Il risultato può essere espresso come un modello di dominio che mostra i concetti o gli oggetti identificativi del dominio.

UML:

La sigla UML sta per (Unified modelling language) è un linguaggio visuale standardizzato utilizzato nell'ingegneria del software per rappresentare, progettare e documentare sistemi software. Il termine "Unified" sta ad indicare che si adatta a qualsiasi piattaforma, processo di sviluppo, dominio applicativo e a qualsiasi linguaggio.

UML può modellare, all'interno del progetto, sia la struttura statica che quella dinamica. In particolare, nel paradigma a oggetti, si definiscono nel seguente modo:

- **Struttura statica:** classi e oggetti (sia di dominio che software) e le loro relazioni.
- **Struttura dinamica:** ciclo di vita degli oggetti e le loro interazioni che realizzano le funzionalità del programma.

Metodi di applicazione di UML:

- **UML come abbozzo:** diagrammi informali, non completi, che descrivono i punti salienti, di interesse e/o complicati del progetto;
- **UML come progetto:** diagrammi relativamente dettagliati utilizzati per il reverse engineering, oppure per la generazione di codice.
- **UML come linguaggio di programmazione:** diagrammi completi e completamente coerenti fra loro, che permettono di generare il codice del software (anche in maniera automatizzata se è stato usato un programma dedicato).

Punti di vista per applicare UML:

- **Punto di vista concettuale:** si modellano concetti o oggetti del mondo reale, scissi dalla loro trasposizione in codice. Le classi concettuali formano il Modello di Dominio;
- **Punto di vista software:** I diagrammi utilizzano la stessa notazione del punto di vista concettuale, descrivono astrazioni o componenti software. Possono rappresentare implementazioni software specifiche a una tecnologia particolare, come Java, oppure specifiche e interfacce dei componenti software, indipendentemente dalla tecnologia utilizzata.

Processi software:

Processi agili e basati sul piano:

- **Processi orientati al piano:** sono processi in cui tutte le attività del processo sono pianificate in anticipo e i progressi sono misurati rispetto a questo piano.
- **Processi agili:** sono processi in cui la pianificazione è incrementale ed è più facile modificare il processo per riflettere le mutevoli esigenze dei clienti.

La maggior parte dei processi pratici include elementi di entrambi gli approcci, quello guidato dal piano e quello agile.

Processi per lo sviluppo del software:

Un processo per lo sviluppo del software definisce un approccio per la costruzione il rilascio e la manutenzione del software. In generale un processo software descrive chi fa che cosa come e quando per raggiungere un certo obiettivo.

Processo software a cascata: il processo software a cascata prevede l'esecuzione sequenziale (a cascata) delle seguenti attività:

- Definizione dei requisiti;
- Design del sistema e del software;
- Implementazione e testing unitario (si vanno a testare le singole componenti software indipendentemente dal sistema)
- Integrazione e testing di sistema (i testa il sistema nella sua interezza come lo utilizzerebbe l'utente).
- Messa in uso e manutenzione (che fa ripartire la cascata da uno degli step precedenti).

Fallisce spesso perché non è flessibile rispetto alle mutevoli esigenze dei clienti; richiede quindi requisiti stabili, non permette modifiche sostanziali. Più il progetto è complesso, più è facile che i requisiti iniziali vengano cambiati. Questo modello è appropriato solo quando i requisiti sono ben compresi e le modifiche saranno piuttosto limitate durante il processo di progettazione.

Sviluppo iterativo, incrementale, evolutivo:

Non è un processo software specifico ma un modello di organizzazione del processo software. Prevede multiple iterazioni di durata temporale fissa (time-boxed, solitamente circa 2-6 settimane) all'interno delle quali si svolgono tutte le attività di processo. Il risultato di ciascuna iterazione è un sistema eseguibile, testato e integrato, anche se parziale. Ciascuna iterazione comprende le proprie attività di analisi dei requisiti, progettazione implementazione e test. Tutto il materiale prodotto servirà da base per l'iterazione successiva (incrementale) e l'iterazione corrente definirà gli obiettivi e raffinerà l'organizzazione dell'iterazione successiva, per meglio adattarsi al progetto (evolutivo). In questo modo i requisiti vengono stabilizzati nel tempo, lasciando spazio soprattutto all'inizio per modifiche anche non banali, portando a una maggior probabilità di successo. La pianificazione è guidata dal rischio (bisogna ridurre i rischi maggiori e stabilizzare quanto prima il nucleo dell'architettura) e dal cliente (si sviluppano prima le caratteristiche per lui prioritarie).

I vantaggi consistono in:

- Elevata flessibilità rispetto alle modifiche;
- Riduzione precoce dei rischi maggiori.
- Progresso visibile fin dall'inizio.
- Feedback precoce, coinvolgimento dell'utente che portano a un sistema che soddisfa meglio le esigenze reali delle parti interessate.
- Migliore gestione della complessità.

Svantaggi:

- Il processo non è visibile.
- La struttura del sistema tende a degradarsi con l'aggiunta di nuovi incrementi.

Unified Process:

Unified Process è un processo di sviluppo software iterativo e incrementale che fornisce una struttura per lo sviluppo del software orientato agli oggetti. È caratterizzato da un approccio guidato dai casi d'uso, centrato sull'architettura e orientato alla gestione del rischio.

Fasi di UP:

Un progetto UP organizza il lavoro e le iterazioni in quattro fasi principali:

- **Ideazione:** visione approssimativa, con stime dei costi e dei tempi;
- **Elaborazione:** realizzazione del nucleo dell'architettura, visione raffinata, identificazione di gran parte dei requisiti;
- **Costruzione:** realizzazione delle capacità operative iniziali e successiva preparazione al rilascio;
- **Transizione:** completamento del prodotto.

Discipline di UP:

Una disciplina è un insieme di attività e dei relativi elaborati in una determinata area, come le attività relative all'analisi dei requisiti. In UP un elaborato è il termine generico che indica un qualsiasi prodotto di lavoro: codice, schemi di base di dati, documenti di testo, diagrammi.

Ci sono diverse discipline in UP in particolare vediamo:

- **Modellazione del business:** l'elaborato modello di dominio per visualizzare i concetti significativi nel dominio dell'applicazione.
- **Requisiti:** gli elaborati modello dei casi d'uso per descrivere i requisiti funzionali e non funzionali.
- **Progettazione:** l'elaborato modello di progetto per il progetto degli oggetti software.

Iterazioni di UP:

Ogni iterazione è una finestra di tempo all'interno della quale si completa un mini-progetto (ad eccezione della prima iterazione, la fase di ideazione, dove non si produce software eseguibile) che include:

- Pianificazione;
- Analisi e progettazione;
- Costruzione;
- Integrazione e test;
- Release (insieme di manufatti previsti e approvati, base per il futuro).

Il prodotto finale è dato da molteplici iterazioni (che si possono sovrapporre in parallelo per grandi squadre). Il tempo dedicato ad ogni disciplina nel "mini-progetto" di un'iterazione varia in base al numero di iterazioni già fatte e alla fase in cui è il progetto: all'inizio si dedica più tempo ai requisiti e più avanti all'implementazione e al testing, per esempio.

Cattive pratiche di UP:

- Si tenta di definire tutti i requisiti all'inizio (cascata);
- Si realizza il nucleo dell'architettura e non lo si testa;
- Si pensa che gli UML siano dettagliati a livello codice (livello di dettaglio linguaggio di programmazione);

- Si pensa che UP consista nel produrre più elaborati possibile;
- Si pensa che ogni fase abbia una sola attività e/o viceversa;
- Si cerca di finire il progetto prima dell'implementazione.

Agile:

Lo sviluppo agile è una forma di sviluppo iterativo che incoraggia l'agilità, ovvero una risposta rapida e flessibile ai cambiamenti.

Metodi Agile:

Lo sviluppo agile è una forma di sviluppo iterativo con interazioni brevi e timeboxed che incoraggiano l'agilità, ovvero una risposta rapida e flessibile ai cambiamenti adottabile da qualsiasi processo iterativo:

Tra i suoi principi troviamo:

- Sviluppo e pianificazione iterativa;
- Consegne incrementali;
- Valori agili: semplicità, leggerezza, valore delle persone, facilità di comunicazione;
- Pratiche agili: modellazione UML solo delle parti complesse del progetto, programmazione a coppie, Test Driven Development, refactoring...

UP agile:

UP Agile è un approccio iterativo e incrementale allo sviluppo software che combina la struttura e la disciplina del Unified Process con la flessibilità e la rapidità delle metodologie agili. Mira a fornire un processo di sviluppo più leggero e adattabile.

Scrum:

Scrum è un metodo agile che consente di sviluppare e rilasciare prodotti software con il più alto valore per i clienti, nel più breve tempo possibile. Scrum si occupa principalmente dell'organizzazione del lavoro e della gestione di progetti. È meno incentrato sugli aspetti tecnici dello sviluppo software lasciando liberi gli sviluppatori di scegliere le tecniche e le metodologie da utilizzare.

Scrum è un approccio iterativo incrementale allo sviluppo del software, ciascuna iterazione, chiamata sprint, ha una durata fissa ed è di tipo timeboxed.

Ci sono vari ruoli:

- **ScrumMaster:** non è necessariamente il project manager; si prefigge di far applicare al meglio il metodo scrum e fa da tramite tra il team e altri team aziendali.
- **Team di sviluppo:** gruppo di sviluppatori responsabili del software e dei vari elaborati;
- **Product owner:** è il proprietario del prodotto, definisce le caratteristiche e specifica le priorità di queste caratteristiche. Il suo obiettivo è massimizzare il valore del prodotto.

La caratteristica distintiva di Scrum tra i metodi agili è l'enfasi sull'adozione di team auto-organizzati e auto-gestiti. Inoltre, Scrum è basato su un insieme di elaborati ed eventi che hanno lo scopo di rendere visibili gli obiettivi e il progresso delle iterazioni e di favorire un adattamento evolutivo del processo di sviluppo.

Ideazione:

In UP l'ideazione è il passo iniziale che permette di definire la visione del progetto. Lo scopo dell'ideazione è quello di raccogliere informazioni per avere una visione comune e decidere se il progetto è fattibile ed è il caso di procedere con un'indagine seria nella fase di elaborazione.

È composta da un'unica iterazione, detta iterazione zero, differente dalle altre perché non punta a creare software eseguibile, ma si concentra sull'avvio di attività ed elaborati. È molto importante notare che nella fase di ideazione viene pianificata la prima iterazione della fase di elaborazione, dove si inizierà a produrre software eseguibile.

Requisiti:

Ogni sistema software deve risolvere uno o più determinati problemi e per far ciò deve fornire un certo numero di funzionalità e possedere alcune caratteristiche di qualità. Un requisito è una capacità o una condizione a cui un sistema, e quindi il progetto, deve essere conforme.

I requisiti devono essere completi rispetto alle richieste del cliente, non ambigui e coerenti fra loro. Ci sono due tipi di requisiti:

- **Requisiti funzionali:** definiscono funzionalità o servizi che il sistema deve fornire, risposte che l'utente si aspetta in determinate condizioni e output attesi a fronte di determinati input. Gli elaborati prodotti in merito sono i Casi d'Uso;
- **Requisiti non funzionali:** I requisiti non funzionali (tutti gli altri requisiti) non riguardano le specifiche funzioni del sistema, ma sono relativi a proprietà del sistema nel suo complesso, come ad esempio sicurezza, prestazioni, scalabilità, usabilità...

I requisiti sono estremamente importanti in un progetto. UP si basa sulla flessibilità: i primi requisiti vengono individuati (insieme al cliente) in prima battuta nell'ideazione e il resto per la maggior parte durante l'elaborazione. Non è da escludere che possano essere modificati più tardi, ma i requisiti modellando con UP tendono a stabilizzarsi con il tempo.

Tipi e categorie di Requisiti:

Nella gestione dei requisiti è utile utilizzare un qualche schema di classificazione come una checklist per la copertura dei requisiti, per ridurre il rischio di non tener conto di qualche importante aspetto del sistema. Una classificazione di uso comune divide i requisiti in categorie in base al modello FURPS (Functional, Usability, Reliability, Performance, Supportability) con il seguente significato:

- **Funzionale:** requisiti funzionali, caratteristiche e capacità funzionali, sicurezza;
- **Usabilità:** riguarda aspetti legati alla facilità d'uso del sistema, documentazione e aiuto per l'utente.
- **Affidabilità:** riguarda caratteristiche come la disponibilità (la quantità di tempo in cui il sistema è attivo e offre servizi agli utenti), la capacità del sistema di tollerare guasti o di poter essere ripristinato a seguito di fallimenti, l'accuratezza.
- **Prestazioni:** riguarda caratteristiche come tempi di risposta, throughput, capacità e uso delle risorse.
- **Sostenibilità:** è l'abilità del sistema software di essere facilmente modificabile per consentire miglioramenti e riparazioni.

Requisiti ed elaborati di UP:

UP offre diversi elaborati dei requisiti, i principali sono i seguenti:

- **Modello dei casi d'uso:** Un insieme di scenari tipici dell'utilizzo di un sistema. Usato principalmente per i requisiti funzionali.
- **Specifiche supplementari:** ciò che non rientra nei casi d'uso.
- **Glossario:** definisce i termini significativi, ha il ruolo di dizionario dei dati, registra i requisiti relativi ai dati.
- **Visione:** riassume i requisiti ad alto livello che sono dettagliati nel modello dei casi d'uso e nelle specifiche supplementari.
- **Regole di business:** descrivono i requisiti e le politiche che trascendono un unico progetto software, sono richieste nel dominio o nel business.

Casi d'uso:

Cosa sono:

I casi d'uso sono storie scritte, ampiamente utilizzati per scoprire e registrare i requisiti. Essi influenzano molti aspetti di un progetto, compresa l'analisi e la progettazione orientata agli oggetti.

Definizione:

Un caso d'uso è una collezione di scenari correlati, sia di successo che di fallimento, che descrivono un attore che usa un sistema per raggiungere un obiettivo specifico.

Modello dei casi d'uso:

Si tratta dell'insieme di tutti i casi d'uso scritti; è un modello delle funzionalità del sistema e del suo ambiente. I casi d'uso sono documenti di testo, non diagrammi, e la modellazione dei casi d'uso è innanzitutto un atto di scrittura di testi, non di disegno di diagrammi. Può comunque includere, opzionalmente, un diagramma UML dei casi d'uso che mostra i nomi dei casi d'uso e degli attori. I casi d'uso non sono orientati agli oggetti; quando li si scrive, non si sta facendo analisi OO.

Casi d'uso come requisiti funzionali:

I casi d'uso sono requisiti, soprattutto requisiti funzionali che indicano cosa deve fare il sistema. In Up e in molti altri metodi moderni, i casi d'uso sono il meccanismo centrale che viene consigliato per la scoperta e la definizione dei requisiti.

Tipi di attori:

- **Attore Primario:** utilizza direttamente i servizi del sistema in discussione affinché vengano raggiunti degli obiettivi utente.
- **Attore finale:** vuole che il sistema in discussione sia utilizzato affinché vengano raggiunti dei suoi obiettivi. Un esempio di attore finale è il cliente, spesso l'attore primario e l'attore finale di un caso d'uso coincidono.
- **Attore di supporto:** offre un servizio al sistema in discussione, un esempio potrebbe essere il servizio informatico di autorizzazione dei pagamenti con carta di credito.
- **Attore fuori scena:** Ha un interesse nel comportamento del caso d'uso, ma non è un attore primario, finale o di supporto.

Formati dei casi d'uso:

I casi d'uso possono essere scritti utilizzando diversi formati e livelli di formalità:

- **Formato breve:** riepilogo conciso di un solo paragrafo, normalmente relativo al solo scenario principale di successo.

- **Formato informale:** più paragrafi scritti in modo informale, relativi a vari scenari.
- **Formato dettagliato:** tutti i passi e le variazioni sono scritti nel dettaglio; Ci sono anche delle sezioni di supporto, come le pre-condizioni e le garanzie di successo.

Livello dei casi d'uso:

I casi d'uso vengono classificati con un livello:

- **Livello di obiettivo utente:** è il tipo più comune, che descrive gli scenari con cui un attore primario può portare a termine il suo lavoro raggiungendo degli obiettivi.
- **Livello di sotto-funzione:** descrive invece dei sotto-passi, richiesti come al raggiungimento di un obiettivo dell'utente.

Casi d'uso a scatola nera:

I casi d'uso a scatola nera sono il tipo più comune e consigliato; non descrivono il funzionamento interno del sistema, i suoi componenti o aspetti relativi al suo progetto. Piuttosto, il sistema è descritto come dotato di responsabilità. Usando i casi d'uso a scatola nera per definire le responsabilità del sistema, è possibile specificare che cosa deve fare il sistema (comportamento o requisiti funzionali) senza decidere come lo farà (progettazione).

Stesura dei casi d'uso:

Per stilare un caso d'uso efficace bisogna:

- Scegliere i confini del sistema (anche attraverso attori esterni);
- Identificare gli attori primari;
- Identificare gli obiettivi di ciascun attore primario;
- Definire i casi d'uso che soddisfano gli obiettivi degli utenti, il nome va scelto in base all'obiettivo.

È necessario scrivere in modo essenziale ma completo non fraintendibile, e concentrarsi sullo scopo reale dell'utente (risultati che per l'utente hanno valore). La narrativa di un Caso d'Uso viene espressa a livello di intenzioni dell'utente e delle responsabilità del sistema, anziché con riferimento ad azioni concrete: questo significa che non è necessario fornire alcun dettaglio implementativo nei Casi d'Uso. Va stilato un Caso d'Uso generalmente per ogni obiettivo utente.

Utilità dei casi uso:

Per verificare l'utilità dei casi d'uso ci sono 3 test:

- **Test del capo:** Se il caso d'uso non supera il test del capo significa che non è fortemente mirato a ottenere risultati il cui valore sia misurabile.
- **Test EBP (processo di business elementare):** si concentra sul valore aziendale misurabile. Si controlla che non sia un singolo passo anzi che sia un'attività eseguibile in una sola sessione di lavoro, in risposta a un evento business che genera valore business osservabile e misurabile.
- **Test della dimensione:** un Caso d'Uso non deve essere composto da un solo "passo", deve avere tra le 3 e le 10 pagine di testo.

Diagramma dei casi d'uso:

UML fornisce una notazione dei diagrammi dei casi d'uso che consente di illustrare i nomi e gli attori dei casi d'uso, nonché le relazioni tra gli stessi. Un diagramma dei casi d'uso rappresenta un ottimo quadro del contesto del sistema; esso costituisce un buon diagramma di contesto, che consiste nel mostrare i confini del sistema, ciò che giace al suo esterno e come esso viene utilizzato.

Le relazioni che i casi d'uso possono avere sono tre:

- **Include:** Un caso d'uso A "include" un caso d'uso B se B è sempre eseguito quando si esegue A. Rappresenta funzionalità comuni utilizzate da più casi d'uso.
- **Extend:** Un caso d'uso A "estende" un caso d'uso B se A aggiunge comportamenti opzionali a B. Rappresenta variazioni o comportamenti condizionali del caso d'uso di base. L'estensione può verificarsi solo in punti specifici (extension points) del caso d'uso base.
- **Generalizzazione:** Simile all'ereditarietà nella programmazione orientata agli oggetti. Un caso d'uso specifico eredita le caratteristiche di un caso d'uso più generale. Permette di definire varianti di un caso d'uso base.

Elaborazione: prima iterazione

L'elaborazione è la serie iniziale di iterazioni durante le quali il team esegue un'indagine seria, implementa (scrivendo il codice e facendo test) il nucleo dell'architettura, chiarisce la maggior parte dei requisiti e affronta le problematiche di alto rischio; è spesso costituita da due o più iterazioni e durante questa fase non vengono creati prototipi "usa e getta"; al contrario, il codice e la progettazione sono parti di qualità per la produzione del sistema finale. Il risultato è un'architettura dimostrabile che funge da fondamento per le iterazioni successive, insieme a un piano più dettagliato per il proseguo del progetto.

Modellazione di dominio:

Modello di dominio:

Un modello di dominio è una rappresentazione visuale di classi concettuali o di oggetti del mondo reale, nonché delle relazioni tra di essi. In UP, il termine di modello di dominio indica una rappresentazione di classi concettuali del mondo reale, non di oggetti software.

Applicando la notazione UML, un modello di dominio può essere realizzato in pratica come uno o più diagrammi delle classi in cui non sono definite operazioni e adottando un punto di vista concettuale.

- **Classe concettuale:** Una classe concettuale rappresenta un'entità o un concetto nel dominio di interesse, ossia un insieme di oggetti con caratteristiche simili;
- **Associazione:** Un'associazione indica una relazione tra gli oggetti di due classi;
- **Attributo:** un attributo rappresenta una proprietà elementare degli oggetti di una classe.

Classi concettuali:

Una classe concettuale è un'idea, un oggetto del mondo reale; formalmente è definita da:

- **Il simbolo** è una parola o un'immagine usata rappresentare la classe concettuale.
- **L'intensione** è la definizione (in linguaggio naturale) della classe concettuale.
- **L'estensione** è l'insieme degli oggetti descritti dalla classe concettuale.

Una classe è un descrittore per un insieme di oggetti che possiedono le stesse caratteristiche, le istanze di una classe sono dette oggetti. In un diagramma delle classi di UML, una classe viene mostrata come un rettangolo suddiviso in più sezioni, la più in alto riporta il nome della classe, la seconda sezione i suoi attributi.

Gli attributi:

Un attributo è una proprietà elementare degli oggetti di una classe; ogni oggetto assegna un valore ai propri attributi. Gli attributi non devono correlare classi concettuali, (non vanno usati come attributi di chiave esterna). Le visibilità si indicano in questo modo: + public, ~ package, # protected, - private. La visibilità di default è privata. La maggior parte degli attributi dovrebbe essere di un tipo primitivo, non complesso: (Boolean, Date, Number, Character, String, Time).

Le associazioni:

Un'associazione è una relazione semantica tra classi, che comporta connessioni tra le rispettive istanze. Le istanze di un'associazione sono dette collegamenti. Un'associazione è rappresentata da una linea, eventualmente orientata per indicare il senso di lettura etichettata con il nome della relazione, che può avere eventualmente dei ruoli e le cardinalità.

Le associazioni speciali offerte da UML sono:

- **Aggregazione:** è un tipo di associazione/relazione intero-parte. L'aggregato può esistere indipendentemente dalle parti e viceversa; le parti possono essere condivise da più aggregati. L'aggregato è incompleto se mancano alcune delle sue parti;
- **Composizione:** la composizione è un tipo forte di aggregazione intero-parte e implica che:
 - Ogni istanza della parte deve appartenere sempre e solamente a una istanza del composto;
 - Il composto ha la responsabilità rispetto alle sue parti;
 - Il composto può rilasciare una delle parti se un altro composto se ne prende la responsabilità.

Operazioni di sistema e diagrammi di sequenza di sistema

Eventi ed operazioni di sistema:

I casi d'uso descrivono il modo in cui gli attori esterni interagiscono con il sistema software che interessa creare. Durante questa interazione, un attore genera degli eventi di sistema, che costituiscono un input al sistema, di solito per richiedere l'esecuzione di alcune operazioni di sistema, che sono operazioni che il sistema deve definire proprio per gestire tali eventi. Più precisamente, in UML un evento è qualcosa di importante o degno di nota che avviene durante l'esecuzione di un sistema. Inoltre, in UML un'operazione rappresenta una trasformazione oppure un'interrogazione che un oggetto o componente può essere chiamato a eseguire.

SSD (diagrammi di sequenza di sistema):

Un SSD è un elaborato che illustra, per un particolare scenario di un caso d'uso, gli eventi generati dagli attori esterni al sistema, il loro ordine e gli eventi inter-sistema. Il testo dei casi d'uso e gli eventi di sistema da essi sottintesi costituiscono l'input per la creazione degli SSD. Viene quindi adottato il ragionamento a scatola nera per il sistema e gli attori esterni.

Relazione tra SSD e casi d'uso:

Un SSD mostra gli eventi di sistema per un solo scenario di un caso d'uso, e può essere generato per ispezione da tale scenario. In particolare, mostra l'attore primario, il sistema in discussione, nonché i passi che rappresentano le interazioni tra il sistema e l'attore. Le interazioni iniziate dall'attore primario nei confronti del sistema sono mostrate come messaggi con parametri. Si noti che un SSD mostra le interazioni implicate dal testo di un caso d'uso, ma non mostra le azioni eseguite dal sistema.

SSD e UP:

Gli SSD fanno parte del Modello dei Casi d'Uso, dato che sono una visualizzazione delle interazioni implicate negli scenari dei casi d'uso. La maggior parte degli SSD viene creata durante l'elaborazione, quando è utile identificare i dettagli degli eventi di sistema per chiarire quali sono le principali operazioni che il sistema deve essere in grado di gestire, scrivere i contratti delle operazioni di sistema e possibilmente supportare le stime.

Contratti delle operazioni di sistema

Contratti:

I contratti delle operazioni usano pre-condizione e post-condizione per descrivere nel dettaglio i cambiamenti agli oggetti in un modello di dominio, come risultato dell'esecuzione di un'operazione di sistema. I contratti delle operazioni possono essere considerati parte del modello dei casi d'uso, poiché forniscono maggiori dettagli dell'analisi sull'effetto delle operazioni di sistema implicate dai casi d'uso.

Struttura di un contratto:

- **Operazione:** Nome e Parametri (firma) dell'operazione
- **Riferimenti:** casi d'uso in cui può verificarsi questa operazione
- **Pre-condizioni:** ipotesi significative sullo Stato del sistema o degli oggetti nel modello di Dominio prima dell'esecuzione dell'operazione. Si tratta di ipotesi non banali, che dovrebbero essere comunicate al lettore.
- **Post-condizioni:** È la sezione più importante, descrive i cambiamenti di stato degli oggetti nel Modello di dominio dopo il completamento dell'operazione.

Post condizioni:

Le post-condizioni descrivono i cambiamenti nello stato degli oggetti nel modello di dominio, motivo per cui questa sezione è la più importante all'interno di un contratto. Non sono azioni, ma osservazioni sullo stato degli oggetti alla fine dell'operazione. Se la sezione è vuota spesso non ha alcun senso scrivere il contratto e aumentare il numero di elaborati.

Esistono tre tipi di post-condizioni:

- Creazione/distruzione di istanze;
- Modifiche dei valori degli attributi;
- Formazione o rottura di collegamenti (istanze di associazione) tra istanze.

Che cos'è un'operazione di sistema:

Le operazioni di sistema sono operazioni che il sistema, considerato come un componente a scatola nera, offre nella sua interfaccia pubblica. Possono essere individuate negli SSD, dato che mostrano eventi di sistema. Un evento di sistema di input implica che il sistema definisca un'operazione di sistema per gestire quell'evento.

Dai requisiti alla programmazione:

Fare la cosa giusta o fare la cosa bene:

Il lavoro successivo all'analisi dei requisiti si basa sul fare la cosa bene, ovvero sul progettare abilmente una soluzione che soddisfa i requisiti per questa iterazione. Nello sviluppo iterativo, ad

ogni iterazione avviene il passaggio da un interesse centrato sui requisiti a uno incentrato su progettazione e implementazione.

Architettura logica:

Architettura software:

Un'architettura è l'insieme delle decisioni significative sull'organizzazione di un sistema software, la scelta degli elementi strutturali da cui è composto il sistema e delle relative interfacce, insieme al loro comportamento specificato dalle collaborazioni tra questi elementi, la composizione di questi elementi strutturali e comportamentali in sottosistemi via via più ampi e lo stile architetturale che guida questa organizzazione.

Architettura logica:

L'architettura logica di un sistema software è l'organizzazione su larga scala delle classi software in pacchetti o sottosistemi e strati. È chiamata architettura logica poiché non vengono prese decisioni su come questi elementi siano distribuiti, sui processi o sui diversi computer fisici in una rete.

Possiamo dividere l'Architettura logica secondo più criteri:

- **Livello (tier):** solitamente indica un nodo fisico di elaborazione;
- **Strato (layer):** una sezione verticale dell'architettura;
- **Partizione (partition):** una divisione orizzontale di sottosistema di uno strato.

Architettura logica a strati:

L'architettura di un sistema software può essere organizzata secondo diversi “stili”. Uno stile comune per l'architettura logica è l'architettura a strati, in cui il sistema è formato da un insieme di elementi chiamati strati. Ciascuno strato è un gruppo a grana molto grossa di classi, pacchetti o sottosistemi, che ha delle responsabilità coese rispetto a un aspetto importante del sistema. Gli strati sono organizzati in modo tale che strati “più alti” ricorrano ai servizi degli strati “più bassi”, mentre normalmente non avviene il contrario.

Gli strati di un'applicazione software comprendono i seguenti:

- **Interfaccia utente o presentazione:** oggetti software per gestire l'interazione con l'utente.
- **Logica applicativa e oggetti di dominio:** oggetti software che rappresentano concetti del dominio che soddisfano i requisiti dell'applicazione, come calcolare il totale di una vendita.
- **Servizi tecnici:** oggetti e sottosistemi di uso generale che forniscono servizi tecnici di supporto, come l'accesso alla base di dati o il logging degli errori. Questi servizi possono essere indipendenti dall'applicazione e riutilizzabili in diversi sistemi.

Esistono due tipi di architetture a strati:

- **A strati stretta:** uno strato può richiamare solamente i servizi di uno strato immediatamente sottostante;
- **A strati rilassata:** uno strato può richiamare servizi di strati più bassi di diversi livelli.

Vantaggi dell'utilizzo degli strati:

L'uso degli strati contribuisce ad affrontare ed evitare diversi problemi:

- Modifiche del codice che si estendono a tutto il sistema: gli strati permettono quasi sempre modifiche locali senza dover modificare altri strati (indipendenza delle modifiche);
- Intrecci tra logica applicativa e UI (separation of concerns);
- Intrecci tra servizi / logica di business con logica applicativa (separation of concerns);

- Accoppiamenti tra diverse aree di interesse (low coupling).

Strato della logica applicativa:

La logica applicativa è lo strato che riguarda gli oggetti di dominio. È tuttavia utile suddividere la logica applicativa in due strati:

- **Strato del dominio:** gli oggetti di dominio;
- **Strato application:** gli oggetti che gestiscono il workflow da e per gli oggetti di dominio, soprattutto dall'UI (strato opzionale ma molto consigliato).

Verso la progettazione a oggetti:

Progettazione:

Gli sviluppatori possono approcciarsi in modi diversi alla progettazione:

- **Codifica:** progettare mentre si codifica, con TTD e refactoring possibilmente.
- **Disegno, poi codifica:** da diagrammi UML a codice.
- **Solo disegno:** uno strumento che converte disegno in codice (in qualche modo).

In un'ottica agile va ridotto il costo aggiuntivo del disegno e modellare per comprendere e comunicare, anziché documentare. Si modella in gruppi, portando avanti più documenti in parallelo.

Modelli statici e dinamici:

Ci sono due tipi di modelli per gli oggetti:

- **Modelli statici:** Diagrammi delle Classi, di package e di deployment (per progettare i package, attributi e firme dei metodi);
- **Modelli dinamici:** Diagrammi di Sequenza, di Comunicazione, delle Macchine a Stati e di Attività (per progettare la logica, il comportamento e corpo dei metodi). Durante la modellazione dinamica verranno applicati i pattern e verrà applicata la progettazione guidata dalle responsabilità.

Diagrammi di interazione:

Diagrammi di interazione:

UML comprende i diagrammi di interazione per illustrare il modo in cui gli oggetti interagiscono attraverso lo scambio di messaggi. Un diagramma di interazione mostra l'interazione tra un insieme di oggetti, basata sullo scambio di messaggi. Ma che cos'è un'interazione? Un'interazione è una specifica di come alcuni oggetti si scambiano messaggi nel tempo per eseguire un compito nell'ambito di un certo contesto. In un diagramma, questo compito è rappresentato da un **messaggio trovato** che dà inizio all'interazione. Se il compito è complesso, l'oggetto interagisce e collabora con altri oggetti, chiamati partecipanti.

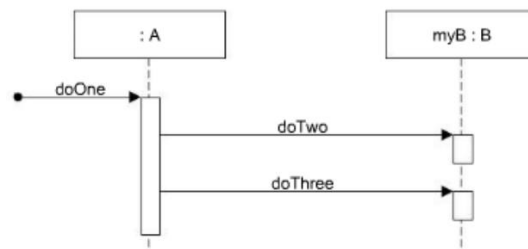
Ci sono due tipi di Diagrammi di Interazione:

- Diagrammi di Sequenza (SD);
- Diagrammi di Comunicazione (CD);

Il termine diagramma di interazione è una generalizzazione di due tipi più specifici di UML, di sequenza e di comunicazione.

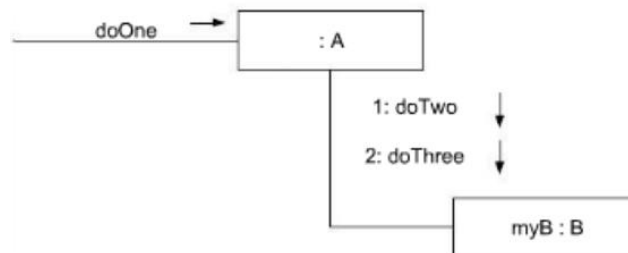
Diagrammi di sequenza:

I diagrammi di sequenza mostrano le interazioni in una specie di formato a steccato, in cui gli oggetti che partecipano all'interazione sono mostrati in alto, uno a fianco dell'altro.



Diagrammi di comunicazione:

I diagrammi di comunicazione mostrano le interazioni tra gli oggetti in un formato a grafo o a rete, in cui gli oggetti possono essere posizionati ovunque nel diagramma.



Operatori dei diagrammi di sequenza:

Operatore	Nome	Significato
opt	Option	C'è un singolo operando che viene eseguito se la condizione è vera (if...then)
alt	Alternatives	Viene eseguito l'operando la cui condizione è vera. Si può usare la parola chiave "altrimenti" o "else" per il caso di default (switch...case)
loop	Loop	Ha una sintassi speciale: loop min, max, [condizione] Esegui loop min volte, poi mentre la condizione è vera esegui loop fino al massimo a max volte in totale
break	Break	Se la condizione di guardia è vera viene eseguito l'operando ma non il resto dell'interazione
ref	Reference	Il frammento combinato fa riferimento ad un'altra interazione

Diagrammi delle classi:

Notazione comune delle classi:

UML comprende i diagrammi delle classi per illustrare le classi, le interfacce e le relative associazioni. Essi sono utilizzati per la modellazione statica degli oggetti.

Diagramma delle classi software:

I diagrammi delle classi possono essere utilizzati, da un punto di vista concettuale, per visualizzare un modello di dominio. Per la discussione occorre anche un termine univoco che chiarisca quando un diagramma delle classi è invece utilizzato da un punto di vista software o di progetto.

Diagrammi degli oggetti software:

Per favorire la comprensione di un diagramma delle classi di progetto, talvolta è utile disegnare un diagramma degli oggetti per descrivere la rappresentazione software di una certa realtà di interesse.

Operazioni:

Una delle sezioni del rettangolo per una classe UML mostra le firme delle operazioni. Un'operazione non è un metodo. Un'operazione di UML è una dichiarazione, con un nome, dei parametri, un tipo di ritorno, un elenco di eccezioni e magari un insieme di vincoli di pre-condizioni e post-condizioni.

Generalizzazione e astrazione:

Una generalizzazione è mostrata in UML come una linea continua e una grossa freccia triangolare dalla sottoclasse verso la superclasse, in particolare è una relazione tassonomica tra un classificatore più generale e un classificatore più specifico. Ogni istanza del classificatore più specifico è anche un'istanza indiretta del classificatore più generale. La generalizzazione equivale all'ereditarietà nei linguaggi di programmazione orientati agli oggetti? Dipende. In un diagramma delle classi da un punto di vista concettuale, ovvero in un modello di dominio, la risposta è no.

Dipendenze:

Le linee di dipendenza possono essere usate in qualsiasi diagramma, ma sono particolarmente comuni nei diagrammi delle classi e dei package. UML comprende una relazione di dipendenza generale che indica che un elemento cliente (di qualsiasi tipo, comprese le classi, i package, i casi d'uso e così via) è a conoscenza di un altro elemento fornitore e che un cambiamento nel fornitore potrebbe influire sul cliente.

Interfacce:

UML offre diversi modi per mostrare l'implementazione di un'interfaccia, il fornire un'interfaccia ai propri clienti (un'interfaccia fornita) e la dipendenza da un'interfaccia (un'interfaccia richiesta). Un'interfaccia è un insieme di funzionalità pubbliche identificate da un nome; separa le specifiche di una funzionalità dall'implementazione della stessa.

Diagrammi di attività e modellazione:

Diagramma delle attività:

Un diagramma di attività di UML mostra le attività sequenziali e parallele in un processo. Tali diagrammi sono utili per la modellazione di processi di business, flussi di lavoro (workflow), flussi dei dati e di algoritmi complessi. Un diagramma di attività offre una notazione ricca per mostrare una sequenza di attività, comprese attività parallele. Può essere applicato a qualsiasi punto di vista o scopo, ma l'applicazione più diffusa è per la visualizzazione dei flussi di lavoro nonché, quindi, dei casi d'uso.

Diagrammi di macchina a stati e modellazione:

Macchine a stati:

Le Macchine a Stati sono dei diagrammi che possono essere utilizzati per modellare il comportamento dinamico di classificatori quali classi, casi d'uso, sottosistemi e interi sistemi. Un diagramma di macchina a stati essenzialmente mostra il ciclo di vita di un oggetto, modellato attraverso una successione di stati (condizione di un oggetto in un lasso di tempo), transizioni (relazioni tra stati etichettate; quando si verifica l'evento l'oggetto cambia stato) ed eventi. Possono anche rappresentare protocolli. UML non li indica come elaborati, vanno realizzati solo se sono utili.

Gli oggetti il cui comportamento è modellato possono essere di due tipi:

- **Indipendenti dallo stato:** risponde sempre in uno stesso modo ad un determinato evento;
- **Dipendenti dallo stato:** risponde in modo diverso in base allo stato in cui si trova.

Rappresentazione di una macchina a stati:

Un Diagramma di Macchina a Stati di UML illustra i possibili stati per un oggetto, ossia le combinazioni dei valori degli attributi, delle sue relazioni e delle attività in corso, attraverso dei rettangoli arrotondati, insieme al comportamento dell'oggetto in termini di transizioni, frecce etichettate con delle azioni istantanee non interrompibili, fra gli stati in risposta agli eventi durante la sua vita. Un ciclo comincia di solito dallo stato iniziale (pallino pieno) e finisce quando arriva nello stato finale (pallino pieno con bordo).

Gli stati:

Uno stato ha un nome, delle azioni di ingresso e di uscita (archi entranti e uscenti), può avere delle transizioni interne (come se fossero transizioni a cappio, non cambiano lo stato) e delle attività interne, che richiedono un tempo finito e sono interrompibili (sono ciò che viene fatto mentre l'oggetto è in quello stato). Si noti dall'immagine la sintassi per ogni tipo di azione/transizione/attività. Esistono anche stati più complessi: parliamo degli stati composti e degli pseudostati.

Nodo azione:

I nodi azione rappresentano unità discrete di lavoro atomiche all'interno dell'attività; quando c'è un token per ogni arco in entrata viene eseguita l'azione del nodo e mandato un token in ogni nodo d'uscita. Un nodo azione può invocare un'attività (nodo rastrello), un comportamento o un'operazione. Eventualmente contiene pezzi di codice dove "self" indica le caratteristiche del contesto dell'attività.

Nodo controllo

I nodi controllo controllano il flusso attraverso l'attività. La gestione dei token varia in base allo specifico nodo.

I nodi oggetto

rappresentano pile/code di token oggetti usati nell'attività. Si può indicare un limite massimo di token, la politica, eventuali collezioni, criteri di selezione o stato dell'oggetto.

Partizioni:

Una partizione è un raggruppamento ad alto livello di azioni correlate; possono essere gerarchiche, verticali o orizzontali, e riferirsi a diverse entità. Se non possono essere mostrate normalmente si può fare con del testo (si indicano le partizioni a cui appartiene il nodo prima del suo nome).

GRASP: progettazione di oggetti con responsabilità:

Progettazione guidata dalla responsabilità:

È importante pensare in termini di responsabilità, ruoli e collaborazioni quando si progetta a oggetti; UP incoraggia fortemente questa pratica. Una responsabilità è un'astrazione di ciò che un oggetto fa o rappresenta; la progettazione guidata dalle responsabilità (RDD, Responsibility-Driven Design) considera un progetto OO come una comunità di oggetti, con responsabilità, che collabora per fornire delle funzionalità:

Le responsabilità sono assegnate alle classi durante la progettazione di oggetti (anche se si deducono già dal Modello di Dominio) e sono di due tipi:

- **Di fare:** (es. eseguire un calcolo o coordinare attività di altri oggetti).
- **Di conoscere:** (es. conoscere i propri dati privati o gli oggetti correlati).

Una responsabilità non è la stessa cosa di un metodo: anzi i metodi sono implementati per adempiere alle responsabilità. I metodi agiscono da soli o collaborano con altri metodi e oggetti per realizzare le responsabilità.

La **RDD** viene fatta iterativamente come segue:

1. Identificare le responsabilità, una alla volta;
2. Chiedersi a quale oggetto software assegnarla;
3. Chiedersi come fa l'oggetto a soddisfarla.

Per l'assegnazione delle responsabilità è utile fare riferimento ai principi definiti dai Pattern GRASP.

Pattern GRASP:

I Pattern GRASP (General Responsibility Assignment Software Patterns) descrivono dei principi di base per la progettazione di oggetti e l'assegnazione di responsabilità. Nell'ambito di UML si può iniziare a pensare alle responsabilità e ad applicare i principi durante il disegno dei Diagrammi di Interazione.

Un pattern è una coppia problema/soluzione ben conosciuta e con un nome (per facilitare comprensione, memorizzazione e comunicazione). Può essere applicato in nuovi contesti, contiene consigli su come applicarlo e discute compromessi, implementazioni, variazioni etc. I pattern si basano su soluzioni e principi già applicati e dimostratisi corretti in diverse situazioni, si fondano quindi sull'esperienza e non sulla sperimentazione.

Di seguito l'elenco dei 9 Pattern GRASP:

- Creator;
- Information Expert;
- Low Coupling;
- Controller;
- High Cohesion;
- Pure Fabrication;
- Polymorphism;
- Indirection;
- Protected Variations

CREATOR:

Problema: chi crea un oggetto A?

Soluzione: Assegna alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera (più sono vere, meglio è; usa come guida i seguenti punti di consiglio):

- B "contiene" o aggrega con una composizione oggetti di tipo A.
- B registra A.
- B utilizza strettamente A.
- B possiede i dati per l'inizializzazione di A.

Altre:

Se la creazione è complessa, usare altre soluzioni come Abstract Factory (pattern GoF).

INFORMATION EXPERT:

Problema: Qual è un principio di base per assegnare responsabilità agli oggetti?

Soluzione: Assegna una responsabilità alla classe che possiede le informazioni necessarie per soddisfarla

Altre: Spesso è necessario far collaborare molti esperti parziali delle informazioni. Un'analogia con il mondo reale: normalmente si assegnano le responsabilità alle persone che posseggono le informazioni necessarie per svolgere un compito. In alcuni casi, le soluzioni suggerite da Information Expert hanno problemi di accoppiamento e di coesione, come il salvataggio in una base di dati: in questo caso occorre mettere la logica della base di dati in un altro luogo, separato dalla logica applicativa.

LOW COUPLING:

Problema: Come ridurre l'impatto dei cambiamenti?

Soluzione: Assegna le responsabilità in modo tale che l'accoppiamento (l'accoppiamento è la misura di quanto un elemento è collegato, conosce o si basa su altri elementi) non necessario rimanga basso. Usa questo principio per valutare le alternative.

Varie:

Le forme di accoppiamento da un tipo X a un tipo Y comprendono:

- La classe X ha un attributo di tipo Y o riferenzia una collezione di oggetti di tipo Y;
- Un oggetto di tipo X richiama operazioni di un oggetto di tipo Y;
- Un oggetto di tipo X crea un oggetto di tipo Y;
- Il tipo X ha un metodo che contiene un parametro/una variabile locale/tipo di ritorno di tipo Y;
- X è una sottoclasse di Y;
- Y è un'interfaccia e X la implementa.

Ha come pattern correlato Protected Variations.

CONTROLLER:

Problema:

Qual è il primo oggetto oltre lo strato UI a ricevere e coordinare ("controllare") un'operazione di sistema?

Soluzione:

Assegna la **responsabilità** a un oggetto che rappresenta una delle seguenti scelte:

- Rappresenta il "sistema" complessivo, un "**oggetto radice**", un dispositivo all'interno del quale viene eseguito il software, un **punto d'accesso al software** o a un sottosistema principale (sono tutte varianti di un facade controller);
- Rappresenta uno scenario di un caso d'uso all'interno del quale si verifica l'operazione di sistema (Use Case Controller, si tratta di una Pure Fabrication).

Altre:

Un controller è il primo oggetto oltre lo strato UI che è responsabile di ricevere e gestire un messaggio di un'operazione di sistema. È semplicemente un pattern di delega, da UI a dominio, e un controller deve limitarsi a coordinare le attività.

Errori comuni:

- Dare troppe responsabilità a un controller (Controller gonfi, cioè controller con coesione bassa);
- Far svolgere parte del lavoro al controller prima di delegarlo;
- Il controller conserva informazioni sul sistema e sul dominio.

Soluzioni:

- Delegare la gestione delle operazioni di sistema;
- Aggiungere altri controller.

HIGH COHESION:

Problema: Come mantenere gli oggetti focalizzati, comprensibili e gestibili e, come effetto collaterale, sostenere Low Coupling?

Soluzione: Assegna le responsabilità in modo tale che la coesione rimanga alta. Usa questo principio per valutare le alternative.

Altre:

La coesione funzionale è una misura di quanto fortemente siano correlate le responsabilità di un elemento. Livelli di coesione:

- **Coesione molto bassa:** una classe responsabile in aree funzionali molto diverse;
- **Coesione bassa:** una classe responsabile di un compito complesso in una sola area funzionale (giustificabile per motivi di efficienza in caso di pochi oggetti server distribuiti che forniscono servizi ampi e poco coesi);
- **Coesione moderata:** una classe ha responsabilità leggere in poche aree diverse, logicamente correlate al concetto rappresentato dalla classe ma non l'una con l'altra;
- **Coesione alta:** una classe ha responsabilità moderate in un'unica area funzionale e collabora con le altre (favorisce il riuso)

PURE FABRICATION:

Problema:

Quale oggetto deve avere la responsabilità quando non si vogliono violare High Cohesion e Low Coupling, o altri obiettivi, ma le soluzioni suggerite da Expert (ad esempio) non sono appropriate?

Soluzione:

Assegnare un insieme di responsabilità altamente coeso a una classe artificiale o di convenienza, che non rappresenta un concetto del dominio del problema, ma piuttosto è una classe inventata per sostenere coesione alta, accoppiamento basso e riuso

Altre:

Errori comuni:

- Utilizzo eccessivo, porta ad una decomposizione del codice (eccesso di oggetti comportamentali senza informazioni richieste per soddisfarli);
- Assegnare un insieme di responsabilità a una classe artificiale senza però sostenere coesione alta, accoppiamento basso e riuso;

POLYMORPHISM:

Problema:

Come gestire alternative basate sul tipo? Come creare componenti software inseribili?

Soluzione:

Assegna la responsabilità per le variazioni di comportamento a diversi tipi che implementano un'interfaccia comune.

Altre:

Le estensioni sono facili da aggiungere ed è possibile introdurre nuove implementazioni senza influire sui client. Tuttavia, non si devono progettare variazioni di fatto improbabili che in realtà non si verificheranno mai.

INDIRECTION:

Problema:

Dove assegnare una responsabilità, per evitare l'accoppiamento diretto tra più elementi? Come disaccoppiare gli oggetti in modo da sostenere un alto riuso e un accoppiamento basso?

Soluzione:

Si assegna la responsabilità ad un oggetto intermediario che media tra altri componenti o servizi, in modo che non ci sia un accoppiamento diretto tra di essi.

Altre:

L'intermediario crea una indirection tra le componenti: si ha accoppiamento più basso tra componenti. Esempio di oggetti intermediari tra strato di dominio e servizi esterni: Adapter (vi è anche polimorfismo tra gli oggetti adattatori).

PROTECTED VARIATIONS:

Problema:

Come progettare oggetti, sottosistemi e sistemi in modo tale che le variazioni o l'instabilità di questi elementi non si riversino su altri elementi?

Soluzione:

Si identificano i punti in cui sono previste delle variazioni, poi si assegnano le responsabilità per creare una "interfaccia" stabile attorno a questi punti.

Altre:

Come per il Polymorphism, le estensioni sono facili da aggiungere ed è possibile introdurre nuove implementazioni senza influire sui client. Inoltre, è possibile ridurre l'impatto/il costo dei cambiamenti. Non utilizzare Protected Variations per variazioni che non si verificheranno mai.

Design pattern GOF (Gang on Four):

I design pattern GOF raccolgono 23 design classificati in base al loro scopo, Ciascun design pattern descrive una soluzione progettuale comune a un problema di progettazione ricorrente. I design pattern GOF sono classificati in base al loro scopo, che può essere **creazionale**, **strutturale** o **comportamentale**. Un pattern creazionale riguarda la creazione di uno o più oggetti; un pattern strutturale riguarda la rappresentazione di informazioni della composizione di classi e oggetti; infine, un pattern comportamentale si occupa dell'interazione tra classi e oggetti per ottenere un certo comportamento.

Tra quelli più usati ci sono:

1. Singleton
2. Adapter
3. Strategy
4. facade
5. Observer
6. Composite
7. Factory

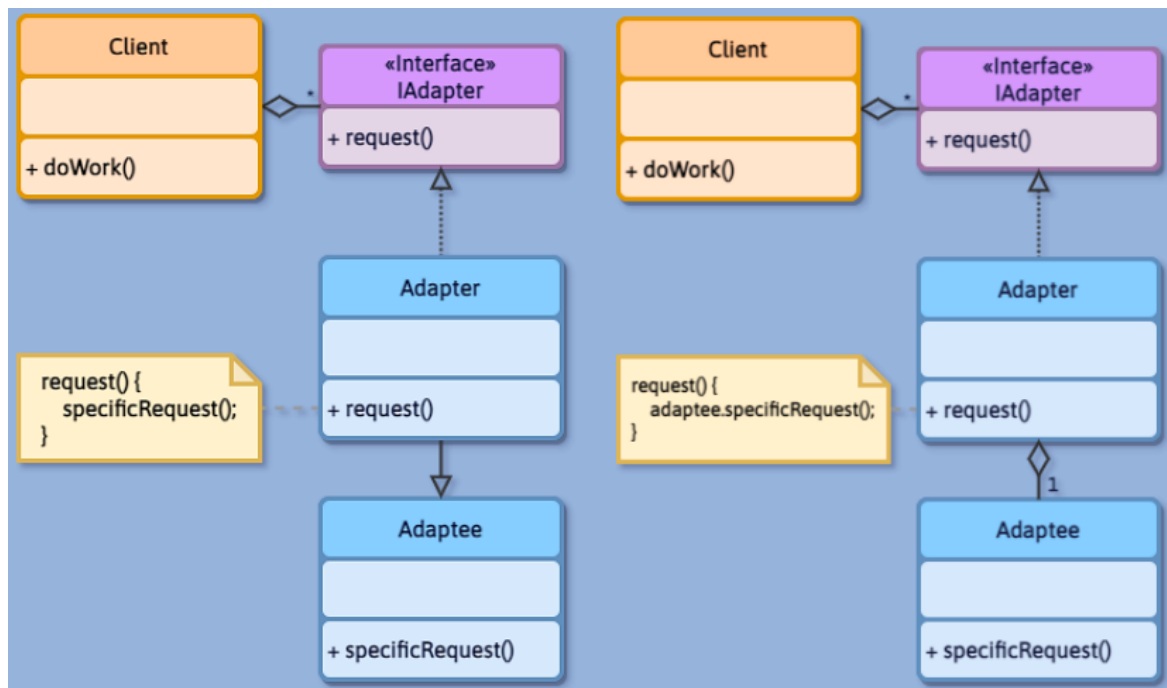
Adapter:

Problema: Come gestire interfacce incompatibili, o fornire un'interfaccia stabile a componenti simili ma con interfacce diverse?

Soluzione: Converti l'interfaccia originale di un componente in un'altra interfaccia, attraverso un oggetto adattatore intermedio.

Ci sono due tipi di Adapter:

1. L'Adapter Classe;
2. L'Adapter Oggetto.

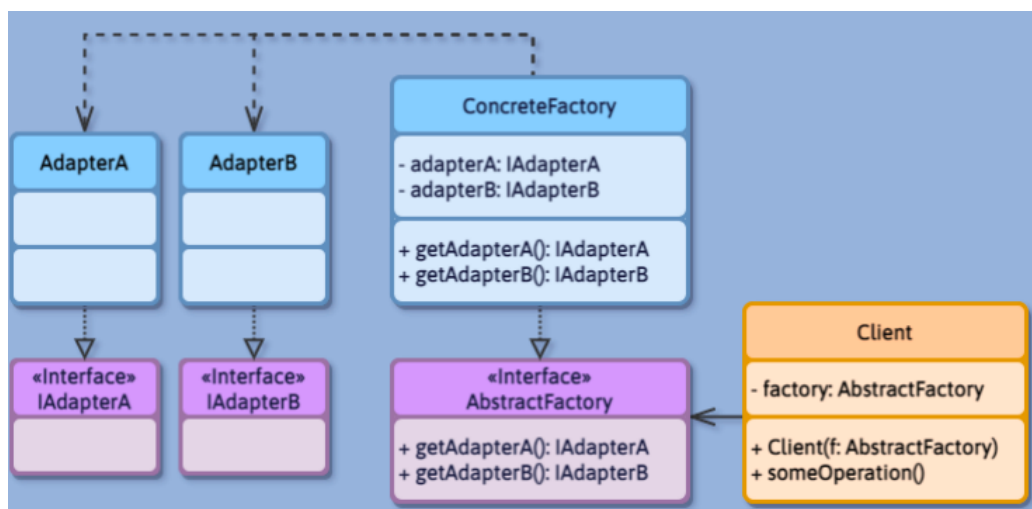


Factory:

Problema: Il pattern Factory risolve il problema della creazione di oggetti senza specificare la loro classe esatta, permettendo la creazione di istanze di classi che condividono un'interfaccia comune, ma delegando la decisione su quale classe concreta istanziare a sottoclassi o a un componente dedicato.

Soluzione: Crea un oggetto Pure Fabrication chiamato “Factory” che gestisce la creazione.

La Factory non è un'interfaccia ma una classe, spesso Singleton; vi sono attributi di cui il tipo è un'interfaccia Adapter e metodi getter che restituiscono una qualsiasi implementazione.

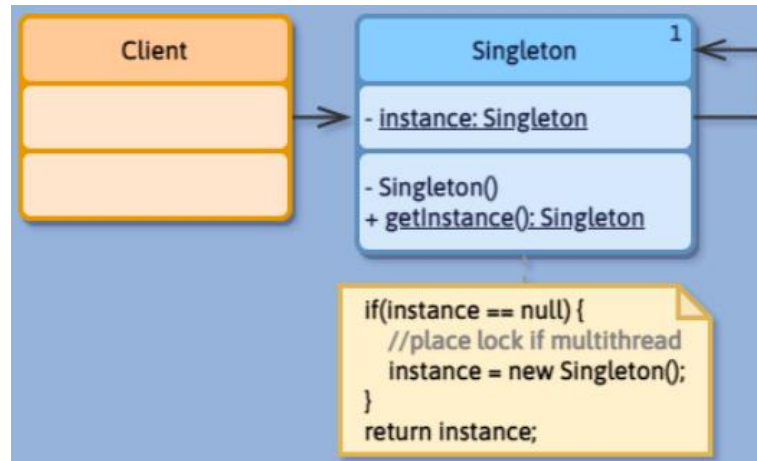


Singleton:

Problema: Il pattern Singleton affronta il problema di garantire che una classe abbia una sola istanza e fornisca un punto di accesso globale ad essa.

Soluzione: Definisce un metodo statico (di classe) della classe che restituisce l'oggetto singleton.

In UML si può scrivere 1 in alto a destra per indicare una classe Singleton. Si riconosce una classe Singleton da un attributo privato statico `instance` chiamato "instance" (o `uniqueInstance`), dal costruttore privato e dal metodo pubblico statico `getInstance` senza parametri. È importante mettere `synchronized` sul metodo `getInstance` per utilizzare un lock e quindi evitare di creare due istanze in caso di thread concorrenti. Non è efficace nel caso di applicazioni distribuite poiché ciascuna VM potrebbe avere un'istanza diversa della classe Singleton. Viene spesso utilizzato per oggetti Factory e Facade.



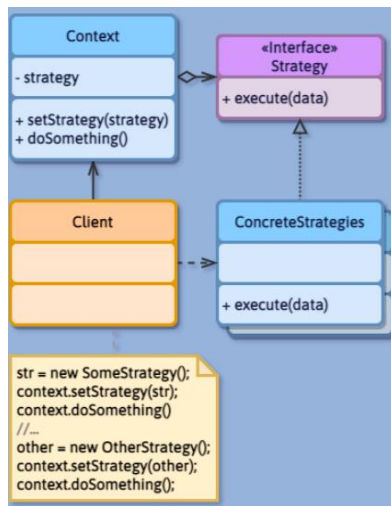
Strategy:

Problema: Il problema del pattern Strategy si riferisce alla necessità di cambiare il comportamento di un algoritmo in fase di esecuzione senza modificare il codice dei client che utilizzano questo algoritmo.

Soluzione: Definisce un'interfaccia comune per tutti gli algoritmi supportati, incapsula ogni algoritmo in una classe separata che implementa questa interfaccia.

Esempio: fornire una logica più complessa per la determinazione del prezzo scontato di una vendita. Leggendo lo schema:

- Context mantiene un riferimento a una delle strategie concrete e comunica con questo oggetto solo mediante l'interfaccia Strategy;
- L'interfaccia Strategy è comune a tutte le strategie concrete;
- ConcreteStrategies indica le classi che implementano variazioni di un algoritmo;
- Context richiama il metodo sull'oggetto strategy associato ogni volta che ha bisogno di eseguire l'algoritmo. Non conosce il tipo dinamico della strategy o come viene eseguito l'algoritmo;
- Il Client crea un oggetto specifico strategy e lo passa al Context. Quest'ultimo offre un setter che permette ai client di sostituire a runtime la strategy associata. Strategy si basa su Polymorphism e fornisce Protected Variations rispetto agli algoritmi variabili. Sono spesso create da una Factory.



Observer:

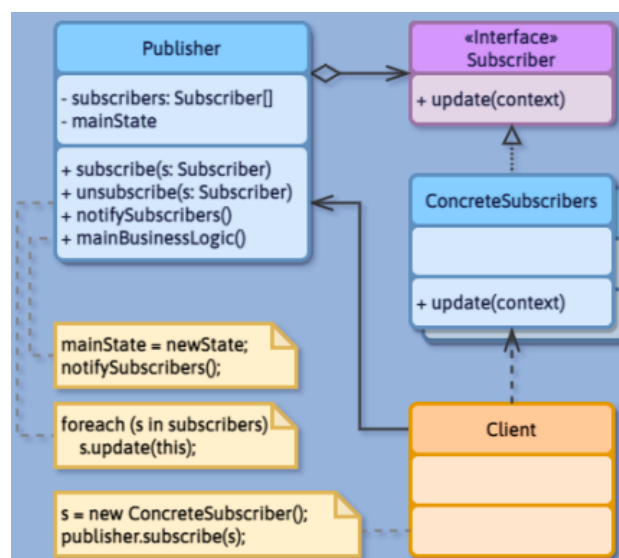
Problema: Diversi tipi di oggetti subscriber sono interessati ai cambiamenti di stato o agli eventi di un oggetto publisher. Ciascun subscriber vuole reagire in un suo modo proprio quando il publisher genera un evento. Inoltre, il publisher vuole mantenere un accoppiamento basso verso i subscriber. Che cosa fare?

Soluzione: Definisci un'interfaccia subscriber o listener. Gli oggetti subscriber implementano questa interfaccia. Il publisher registra dinamicamente i subscriber che sono interessati ai suoi eventi, e li avvisa quando si verifica un evento.

Fornisce un modo per accoppiare in modo debole gli oggetti che devono comunicare, in quanto:

- I publisher conoscono i subscriber solo mediante un'interfaccia;
- I subscriber possono registrarsi/cancellarsi dinamicamente con il publisher.

Qui è il client a creare e aggiornare Publisher e Subscriber. Observer si basa su Polymorphism e fornisce Protected Variations in termini di protezione del publisher dalla conoscenza della classe specifica degli oggetti con cui comunica.



Composite:

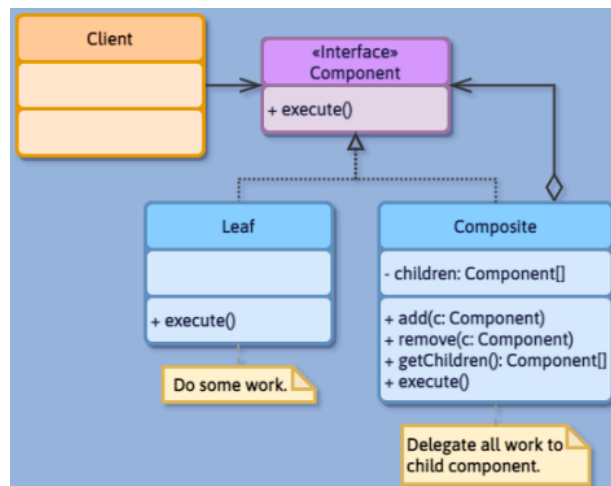
Problema: Come trattare un gruppo o una struttura composta di oggetti (polimorficamente) dello stesso tipo nello stesso modo di un oggetto non composto (atomico)?

Soluzione: Definisce le classi per gli oggetti composti e atomici in modo che implementino la stessa interfaccia.

Leggendo lo schema:

1. Definisce un'interfaccia comune (Component) per tutti gli oggetti nella composizione, sia semplici che composti.
2. Crea una classe foglia (Leaf) per rappresentare gli oggetti semplici.
3. Crea una classe composta (Composite) che può contenere altri componenti (foglie o altri composti).
4. Permette ai client di trattare oggetti individuali e composizioni di oggetti in modo uniforme.

Composite è spesso utilizzato con Strategy, si basa su Polymorphism e fornisce Protected Variations su un client.

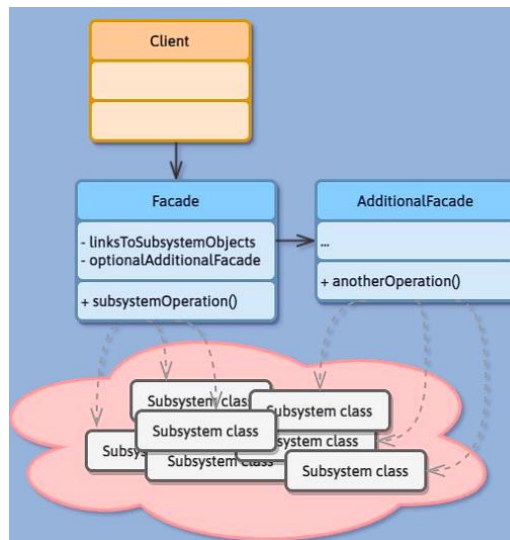


Facade:

Problema: È richiesta un'interfaccia comune e unificata per un insieme disparato di implementazioni o interfacce, come per definire un sottosistema. Può verificarsi un accoppiamento indesiderato a molti oggetti nel sottosistema, oppure l'implementazione del sottosistema può cambiare. Che cosa fare?

Soluzione: Definisce un punto di contatto singolo con il sottosistema, ovvero un oggetto facade (facciata) che copre il sottosistema. Questo oggetto facade presenta un'interfaccia singola e unificata ed è responsabile della collaborazione con i componenti del sottosistema.

Si tratta di un oggetto front-end che rappresenta il punto di entrata singolo ai servizi di un sottosistema. Normalmente si accede a un Facade attraverso il pattern Singleton. Fornisce Protected Variations dall'implementazione di un sottosistema, aggiungendo un oggetto Indirection che aiuta a sostenere Low Coupling. Un facade controller è un tipo Facade.



Progettare per la visibilità:

Progettare la visibilità:

La visibilità è la capacità di un oggetto di "vedere" o di avere un riferimento ad un altro oggetto. Se un oggetto mittente desidera inviare un messaggio a un oggetto destinatario, allora deve avere un qualche tipo di riferimento a quest'ultimo.

La visibilità può essere ottenuta dall'oggetto A all'oggetto B in quattro modi:

- Visibilità per attributo (B è un attributo di A);
- Visibilità per parametro (B è un parametro di un metodo di A);
- Visibilità locale (B è un oggetto locale di un metodo di A);
- Visibilità globale (B è in qualche modo visibile globalmente).

La visibilità per attributo e globale sono relativamente permanenti, mentre quella per parametro e locale sono relativamente temporanee poiché persistono solo nell'ambito del metodo.

Trasformare i progetti in codice:

Dalla progettazione all'implementazione:

Gli elaborati creati durante la disciplina Progettazione sono utilizzati come input per il processo di generazione del codice (disciplina dell'Implementazione).

L'Implementazione in un linguaggio Object Oriented richiede la scrittura di codice per:

- Definire classi e interfacce;
- Dichiarare variabili di istanza;
- Definire metodi e costruttori.

Sviluppo guidato dai test e refactoring:

Il TDD (test driven development) è una best practice basata sulla produzione del codice di test prima del codice da verificare, lo sviluppatore scrive il codice dei test unitari per quasi tutto il codice di produzione. I vantaggi di questa tecnica sono i seguenti:

- I test vengono effettivamente scritti;
- Scrivendo prima i test il programmatore ha già un'idea più chiara di quello che il programma dovrà fare ed ha più chance di successo, la prende come una sfida; nel momento in cui si testa

il programma e i test passano, il programmatore si sente soddisfatto ed è più motivato a proseguire;

- Chiarezza dell'interfaccia e del comportamento dettagliati;
- Verifica automatica, ripetibile e dimostrabile;
- Fiducia nel cambiare le cose.

Tipologie di test:

- **Test unitari:** hanno lo scopo di verificare il funzionamento delle piccole parti ("unità") del sistema, ma non di verificare il sistema nel suo complesso.
- **Test di integrazione:** per verificare la comunicazione tra specifiche parti (elementi architetturali) del sistema.
- **Test di sistema:** per verificare il collegamento complessivo tra tutti gli elementi del sistema.
- **Test di accettazione:** hanno lo scopo di verificare il funzionamento complessivo del sistema, considerato a scatola nera e dal punto di vista dell'utente, ovvero con riferimento a scenari di casi d'uso del sistema.

Test unitari:

Un'unità è una piccola parte verificabile di un'applicazione o di un sistema, dotata di funzionamento autonomo. L'obiettivo di un test unitario è di verificare che l'unità funzioni correttamente. Un metodo di test unitario è logicamente composto da quattro parti:

- **Preparazione:** creare l'oggetto da verificare (quest'ultimo chiamato fixture) e preparare altri oggetti o risorse necessari per l'esecuzione del test;
- **Esecuzione:** far eseguire operazioni alla fixture, richiedendo lo specifico comportamento da verificare;
- **Verifica:** valutare che i risultati ottenuti corrispondano a quelli previsti;
- **Rilascio:** opzionalmente rilascia o ripulisce gli oggetti e le risorse utilizzate.

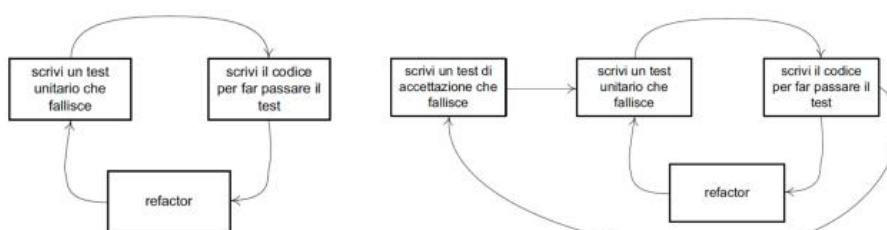
Tipi di cicli per test unitari:

Il TDD si basa su cicli di lavorazione molto brevi, guidati dalle seguenti regole:

- Scrivere un test unitario che fallisce, per dimostrare la mancanza o di una funzionalità o di codice;
- Scrivere il codice più semplice possibile per far passare il test;
- Riscrivere o ristrutturare il codice, migliorandolo, oppure passare a scrivere il prossimo test unitario.

Esistono due tipi di ciclo nel TDD:

- **Ciclo di base:** è quello descritto in precedenza.
- **Ciclo doppio:** In quello doppio ci sono i cicli di lavorazione brevi relativi ai test unitari, mentre il ciclo più esterno riguarda i test di accettazione.



Il ciclo di base del TDD per i test unitari.

Il doppio ciclo del TDD.

Refactoring:

Il refactoring è un metodo strutturato e disciplinato per riscrivere o ristrutturare codice esistente senza però modificarne il comportamento esterno, applicando piccoli passi di trasformazione in combinazione con la ripetizione dei test ad ogni passo. L'essenza del refactoring è applicare piccole trasformazioni che preservano il comportamento, una alla volta. Dopo ciascuna trasformazione, i test unitari vengono eseguiti nuovamente per dimostrare che il refactoring non abbia provocato una regressione (un fallimento). Tutti i test unitari sostengono il processo di refactoring.

I vantaggi del refactoring riguardano:

- **Miglioramento continuo del codice:** è importante migliorare la struttura del codice dopo che è stato scritto, per evitare che diventi disordinato;
- **Preparazione al cambiamento:** il refactoring consente di preparare il codice all'introduzione di nuove funzionalità, nonché all'applicazione di cambiamenti.

Gli obiettivi del refactoring sono i seguenti:

- Eliminare il codice duplicato;
- Migliorare la chiarezza;
- Abbreviare i metodi lunghi.

Il codice su cui è stato eseguito un buon refactoring è breve, conciso, chiaro e senza duplicazioni. Un codice che non ha queste qualità si dice che abbia un "cattivo odore". I cattivi odori del codice (code smell) sono una metafora del refactoring; sono indicazioni che qualcosa nel codice potrebbe non essere corretto.

I code smell si possono suddividere in più categorie:

- **Bloaters:** metodi, classi o in generale codice sovradimensionato.
 - Long Method
 - Large Class
 - Long Parameter List
- **Object-Orientation Abusers:** implementazioni errate, incomplete o deboli dei principi della programmazione OO:
 - Switch Statement
 - Refused Bequest
- **Change Preventers:** parti di codice altamente accoppiate che impediscono cambiamenti rapidi:
 - Shotgun Surgery
- **Dispensables:** codice inutile, che sarebbe meglio togliere o integrare in altre parti per rendere il codice più pulito efficiente e comprensibile:
 - Duplicated Code
 - Data Class
 - Comments
- **Couplers:** classi che hanno eccessivi accoppiamenti o classi che delegano eccessivamente:
 - Future Envy

B L O A T E R S

Code Smell	Sintomo	Condizioni	Refactoring
Long Method	Un metodo ha più di 10 linee di codice	Se si può ridurre senza problemi la lunghezza	Extract Method
		Se ci sono problemi con variabili locali	Replace Temp with Query
			Introduce Parameter Object
			Preserve Whole Object
		Se non funzionano i precedenti	Replace Method with Method Object
		Se ci sono condizionali	Decompose Conditional
		Se ci sono dei cicli	Extract Method
	Large Class	Una classe ha troppi metodi	Extract Class
		Se parte del comportamento può essere implementato in modo diverso o viene usato solo in pochi casi	Exrtact Subclass
		Se uno o più metodi si possono coerentemente spostare in altre classi	Move Method
Long parameter List	Un metodo ha troppi parametri	Se alcuni dei parametri sono risultati di chiamate a metodi di un altro oggetto	Replace Parameter with Method Call
		Se più parametri sono relativi a un oggetto	Preserve Whole Object
		Se i parametri non sono correlati	Introduce Parameter Object
Switch statement	Si ha uno switch molto lungo o più if...else	Se si può sfruttare il polimorfismo	Replace Conditional with Polymorphism
		Se si può isolare lo switch e	Extract Method
			Move Method

O O A B U S E R S			assegnarlo ad una sottoclasse	
	Refuses Bequest	La sottoclasse non usa tutti i metodi e le proprietà ereditate, la gerarchia non è corretta	Se l'ereditarietà non ha molto senso	Replace Inheritance with Delegation
			Se l'ereditarietà è appropriata e si possono togliere metodi e parametri dalla superclasse per metterli dentro a una nuova superclasse	Extract Superclass
C H A N G E P R E V E N T E R S	Shotgun Surgery	Per modificare il codice vanno apportate minime modifiche a molte classi diverse	Se si può riorganizzare il comportamento raggruppando gli elementi in un'altra classe	Move Method
				Move Field
			Se la precedente procedura lascia classi quasi vuote	Inline Class
D I S P E N S A B E S	Duplicated Code	Lo stesso codice appare in molti luoghi (>2)	Se ha senso rendere il codice un metodo chiamabile	Extract Method
	Data Class	Classe che funge solo da contenitore per i dati	Se si possono integrare nella Data Class i metodi usati altrove per manipolare i dati	Move Method
				Extract Class
	Comments	Troppi commenti	Se il commento spiega un'espressione lunga	Extract Variable
			Se il commento spiega una porzione di codice	Extract Method
			Se il metodo è già estratto ma	Rename Method

			molto commentato	
			Se il commento afferma regole necessarie per il corretto funzionamento del sistema	Introduce Assertion
C O U P L E R S	Feature Envy	Un metodo fa accedere ai dati di un altro oggetto più che ai propri dati	Se il metodo dovrebbe stare chiaramente da un'altra parte (usa tanti dati di un'altra classe)	Move Method
			Se solo una parte del metodo accede ai dati	Extract Method
			Se tutto il metodo usa tante funzioni di tante classi, e si può “smembrare” per poi suddividerlo tra le stesse	Move Method
				Extract Method